

This article was downloaded by: [ECU Libraries]

On: 21 April 2015, At: 11:20

Publisher: Routledge

Informa Ltd Registered in England and Wales Registered Number: 1072954 Registered office: Mortimer House, 37-41 Mortimer Street, London W1T 3JH, UK



Journal of Management Information Systems

Publication details, including instructions for authors and subscription information:

<http://www.tandfonline.com/loi/mmis20>

Service-Oriented Methodology for Systems Development

Mark Keith^a, Haluk Demirkan^a & Michael Goul^b

^a Marriott School of Management, Brigham Young University

^b W.P. Carey School of Business, Arizona State University

Published online: 08 Dec 2014.

To cite this article: Mark Keith, Haluk Demirkan & Michael Goul (2013) Service-Oriented Methodology for Systems Development, Journal of Management Information Systems, 30:1, 227-260

To link to this article: <http://dx.doi.org/10.2753/MIS0742-1222300107>

PLEASE SCROLL DOWN FOR ARTICLE

Taylor & Francis makes every effort to ensure the accuracy of all the information (the "Content") contained in the publications on our platform. However, Taylor & Francis, our agents, and our licensors make no representations or warranties whatsoever as to the accuracy, completeness, or suitability for any purpose of the Content. Any opinions and views expressed in this publication are the opinions and views of the authors, and are not the views of or endorsed by Taylor & Francis. The accuracy of the Content should not be relied upon and should be independently verified with primary sources of information. Taylor and Francis shall not be liable for any losses, actions, claims, proceedings, demands, costs, expenses, damages, and other liabilities whatsoever or howsoever caused arising directly or indirectly in connection with, in relation to or arising out of the use of the Content.

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden. Terms & Conditions of access and use can be found at <http://www.tandfonline.com/page/terms-and-conditions>

Service-Oriented Methodology for Systems Development

MARK KEITH, HALUK DEMIRKAN, AND MICHAEL GOUL

MARK KEITH is an assistant professor of information systems, Marriott School of Management, Brigham Young University. He received his Ph.D. in information systems from Arizona State University. His recent research interests concern IT project methodology, the service-oriented paradigm, and information privacy with mobile technologies. His research has appeared, or is forthcoming, in the *Journal of Management Information Systems*, *Journal of the Association for Information Systems*, *International Journal of Human-Computer Studies*, *Decision Support Systems*, *Communications of the Association for Information Systems*, *INFORMS Decision Analysis*, and other leading journals and conference proceedings.

HALUK DEMIRKAN is an associate professor of service science, information systems, and supply chain management at the Milgard School of Business, University of Washington-Tacoma. He received his Ph.D. in information systems and operations management from the University of Florida. His recent research interests are in the areas of service science, innovation and ecosystem with analytics. He has more than 15 years of professional work experiences on process innovations with service-oriented solutions and data, analytics, and business intelligence for Fortune 100 companies. He is a track chair for the Analytics, Mobile, and Service Science Track at the Hawaii International Conference on System Sciences (HICSS) and a board of director for the International Society of Service Innovation Professionals.

MICHAEL GOUL is a professor of information systems and chair of the Department of Information Systems and a research faculty member of the Center for Services Leadership at the W.P. Carey School of Business at Arizona State University. He received his Ph.D. from Oregon State University. His recent research interests are in the areas of service computing, smart and self-service, and advanced analytics. He has served as journal editor, special issue editor, Association for Information Systems (AIS) vice president, AIS conference and program chair, and chair of the AIS Special Interest Group on Decision Support, Knowledge and Data Management Systems (SIGDSS). Dr. Goul recently held a sabbatical appointment as a William J. Clinton Distinguished University Fellow at the newest presidential school, the Clinton School of Public Service, and he the executive director of the Teradata University Network.

ABSTRACT: Despite advances in software development practices, organizations struggle to implement methodologies that match the risk in a project environment with needed coordination capabilities. Plan-driven and agile software development methodologies each have strengths and risks. However, most project environments cannot be classified as entirely “risky” or “stable,” suggesting the need for hybrid approaches. We leverage a design science approach to implement a novel hybrid methodology based on concepts from the service-oriented paradigm. We motivate the approach

using theory on interdependence and coordination, and design the methodology using theory on modularity and service-dominant logic. We also examine the effects of its adoption at a large electrical power company over a three-year period. The results imply that service-oriented theory should be applied to the human processes involved in systems development in order to achieve better fit between project risk, interdependencies, and the selected methodology(ies) in order to improve overall project performance.

KEY WORDS AND PHRASES: design science, modularization, project management, service orientation.

REPORTS OF FAILED INFORMATION TECHNOLOGY (IT) PROJECTS ARE EASY TO FIND. The UK's Integrated Offender Management System, at a cost of almost \$375 million, is still not complete, Canadian WestJet Airline's new passenger reservations system went \$34 million over budget, and Microsoft's estimated \$1 billion Kin mobile phone project resulted in its withdrawal from the market after just two months [18].

The conclusion drawn from these failures is that the project managers (PMs) failed to manage project risks effectively [16, 43] and to match the most appropriate methodology to the task [7, 8]. For example, traditional plan-driven methodologies (i.e., waterfall) assume that most project risks can be managed up front. However, this is not the case with many IT projects. Agile software development (ASD) methodologies (e.g., eXtreme Programming, Scrum) have gained increased interest among researchers and practitioners [26]. To combat the high costs of change associated with plan-driven methodologies, agile methods implement short iterations of planning, analysis, development, testing, and feedback. Project teams using agile methodologies plan on changes taking place throughout the project and therefore minimize the amount of up-front project planning and analysis. As a result, ASD can potentially reduce the project loss caused by changing scope. Plan-driven methodologies are best suited when the project environment is relatively stable, whereas agile methodologies are more appropriate when the risk for scope change is relatively greater [26].

The problem with this dichotomy is that most IT projects are neither mostly "risky" nor "unrisky" but lie somewhere in between. Depending on a variety of factors, certain portions of an IT project can be fairly routine while others are highly uncertain. However, neither methodology is optimized for handling both the risky and unrisky portions of an IT project with peak efficiency.

Further exacerbating this problem is the fact that the appropriate infrastructures for each methodology are quite different. For example, although ASDs have been successful in many organizations (e.g., [48]), they rely on certain assumptions in order to achieve success [53]. Less up-front project planning means that more of the project task interdependencies must be managed during project execution in the form of ad hoc, informal communication among team members. To facilitate this "on-the-fly" coordination adequately, project teams generally require strong knowledge-sharing networks and high levels of expertise [53]. If there is high turnover in project teams or if subject matter experts leave the company, the organization may decide to support

knowledge sharing through a formal codified knowledge management system or some other mechanism that is less ideal for agile methodologies. Extracting from information processing theory, mis-fits between the information processing requirements of a task and the information processing capabilities of the supporting infrastructure cause inefficiencies [15].

There is a need for methodologies that can thrive in hybrid environments as well as a need for new theoretical directions on how to realize an appropriate infrastructure to support these techniques. In recent years, the concept of service orientation as a paradigm for marketing logic [56], technical infrastructures [33], application design [13], resource allocation, and business processes has gained momentum [10] and presents a potential solution to the methodology problem described above. From an IT perspective, this is because of the potential that service-oriented architectures (SOAs) offer for greater organizational agility in environments with high risk [13]. However, although advancements have been made in SOAs and service-computing technologies, there remain inadequate methodologies to tie them all together [27]. This is not to say that methodologies for developing service-oriented systems do not exist (e.g., [41]). Rather, the coordination of people working across processes as supported by a service-oriented collaboration capability remains underaddressed [27].

The purpose of this research is to identify and test design principles for a service-oriented methodology. The key to a successful hybrid approach is that it must be able to accurately modularize the IT project process along natural breakpoints between high- and low-risk activities [8] so that the appropriate resources, technologies, and coordination techniques can be applied to each task [15]. This approach outlined in this research, referred to as *service-oriented methodology for systems development* (SoMSD),¹ is based on principles of modularity [2, 42, 52] and service orientation [14, 56]. The principles include modularizing the software development process into the basic activities of individuals and groups [23], storing standardized service definitions and resource requirements for each modular activity [6], and storing choreographies of these human service activities in a format that can be shared and reused during the planning and analysis stages of future projects [17]. The SoMSD integrates these existing techniques—supported by a human-based SOA [6]—into a high-level methodology to demonstrate the hybrid use of both plan-driven and agile techniques in the same project life cycle based on the different interdependencies found in each human-based service.

This research also draws from the principles of design science and relevant theory established by prior contributions [20, 21, 25, 40]. Whereas behavioral science research generates theories of human and organizational behavior, design science research is concerned with the development and subsequent performance of IT artifacts such as hardware and software technologies, algorithms, user interfaces, process models, and methodologies [25]. In the design science paradigm, theory is generated as the knowledge contained within an artifact. It is theory about “how to do something” [21]. This research follows Peffers et al.’s [40] outline of a methodology for conducting and evaluating design science research, including (1) problem identification, (2) solution objectives, (3) artifact design, (4) demonstration, (5) evaluation, and (6) communication.

The problems with current software project methodologies and the objectives for a solution are expanded in the following section and supported by Thompson's [49] theory on interdependence and coordination and Galbraith's [15] information processing theory. A description of the SoMSD is provided based on theories from the literature on service orientation and modular systems. Then the effectiveness and effects of the SoMSD are demonstrated by an examination of its adoption in the IT unit of a large electrical power company. Finally, we review a post hoc examination of unintended consequences, the limitations of the SoMSD and its practical implications, implications for theory including propositions, and suggestions for future research and practice.

Review of Current Practices

MOST SOFTWARE DEVELOPMENT METHODOLOGIES are identified as being either plan driven, such as the traditional systems development life cycle (SDLC) (i.e., waterfall), or agile based, which depend less on a formal plan and more on ad hoc adjustment throughout the project life cycle. However, in practice, many methodologies fall somewhere in between [58].

Project risks can be divided into two categories: (1) the pre-design stage risk, which exists because of inadequate requirements analyses, and (2) the post-design stage risk, unrelated to the initial plan and analysis [16]. Plan-driven methods assume that most project requirements can be specified up front, before design and development begins. A meticulous process is followed to assure the quality of project requirements, plans, and resources. As a result, plan-driven methods are ideal for managing the pre-design stage risk. In contrast, agile methods assume that project requirements will change. Their success hinges on (1) being able to effectively manage changes as they arise and (2) the ability of project members to coordinate ad hoc throughout the project life cycle. Therefore, agile methods are ideal for post-design stage risk.

Because of this, each methodology requires a different set of coordination mechanisms that are designed to manage and reduce risk [30]. For example, plan-driven methods primarily make use of *formal* coordination mechanisms [37]. Project plans are detailed to a very granular level, the team structure is rigid with a formal hierarchy, and the roles rarely change. Alternatively, ASDs draw primarily on *informal* coordination mechanisms such as ad hoc communication during project execution [11]. PMs will design the team structure (e.g., functional versus divisional versus cross-functional), plans (short term versus long term), and role assignments in a way that will support or emphasize either formal or informal coordination.

Problem Description

The description above explains why most scholars view the issue of methodology selection as a contingency problem [3, 9] in which the most appropriate methodology in a given situation depends on characteristics of the task environment such as risk or uncertainty. Thompson's [49] theory on interdependence explains this contingency problem well, as follows: *Pooled* interdependencies refer to the high-level dependency

each IT project team member shares with all the rest. The PM depends on his or her team members to show up and get their jobs done. Each team member depends on other team members to complete their portions of the project so that the entire project will be successful. Pooled interdependencies can be easily coordinated by establishing *standards* such as the number of work hours per week, rules for collegiality, and other general project guidelines.

Sequential interdependencies are derived from the serial nature of workflow. There is an order, or sequence, in which IT project work must be accomplished. Analysis must happen before design, design before development, development before testing, and so on. Sequential interdependencies exist at even very granular levels of workflow. For example, application code routines must be created before other classes and routines can call them. Regardless of the level of abstraction, all sequential interdependencies can be managed through detailed *planning*. Although planning is more time-consuming and costly than standardization, it can be centralized among a small number of people (e.g., PM and subgroup leaders) to minimize costs.

Reciprocal interdependencies are the result of pairs of team members or groups who are jointly dependent on each other. In an IT project team, these dependencies regularly exist. For example, Programmer A creates a software module that must be used by Programmer B. If A's code does not work correctly, B will need to inform A and ask for a rework. Similarly, if the scope is changed midproject, if resources are lost or changed midproject, or if any other manner of project risk manifests itself, the serial nature of workflow can be disrupted, and reciprocal interdependencies will exist. Basically, reciprocal interdependencies exist when the sequence of workflow cannot be determined beforehand. As a result, the only way to manage reciprocal interdependencies is through *mutual adjustment*, which refers to the ad hoc, informal coordination that takes place during project execution. Mutual adjustment is the most costly form of coordination because it requires that every individual or group involved be able to freely communicate until the work is finished. This requires that interdependent parties either be collocated or provided with sufficient and appropriate communication and collaboration tools.

Interdependence theory posits that IT project teams that have mostly sequential interdependencies should adopt plan-driven software development methodologies, whereas teams with primarily reciprocal interdependencies should adopt agile methodologies [49]. Figure 1 shows the theoretical contingency according to Thompson [49].

The problem is that many software development units cannot be characterized as having mostly reciprocal versus sequential interdependencies. Rather, different segments of the IT project may demonstrate one versus the other [4]. As a result, there is a need for hybrid methodologies that support accurate, long-range planning for some portions of the IT project and support informal, ad hoc collaboration for other portions [8].

Solution Objectives

Based on the above theoretical background [49] and recent research [4], an effective hybrid methodology should meet two high-level objectives: (1) accurately modularize

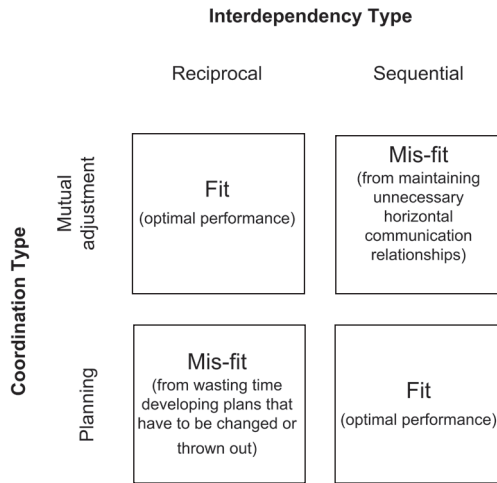


Figure 1. Interdependency Contingency Theory

the software development process along natural breakpoints that divide the tasks with predominantly sequential interdependencies from those that include mostly reciprocal interdependencies, and (2) assign the methodology and resources appropriate to each type of task.

Objective 1: Modularization

Objective 1 is not new. The concept of modularity in both systems architectures and development methodologies has been emphasized in research and practice for many years [1, 2, 60]. For example, PMs using plan-driven methodologies will typically generate some type of work breakdown structure (WBS) [46] during the planning phase. Creating the WBS involves modularizing the project tasks and assigning people and resources to each task. In practice, the knowledge and information required to generate the WBS typically comes from a PM's experience or is based on searching through the WBSs of a handful of past projects [46]. There is relatively little research on practical systems designed to help PMs generate accurate and detailed WBSs [19]. The ability to fully understand the nature of task interdependencies and modularize the project process with respect to interdependencies is exactly what will help PMs to accurately modularize their project process and minimize their coordination costs [49]. Therefore, effective hybrid methodologies should offer a theory-reinforced approach to modularizing the project process.

Objective 2: Methodology and Resource Assignment

The second objective—employing either plan-driven or agile methodologies to each of the modularized, independent tasks within the same project—is much less common

than Objective 1. First, it would require that the PM and team members have enough expertise to be able to effectively use both methodologies. Second, employing multiple project methodologies can be problematic because of the difficulty in changing entrenched human behaviors even when mandated to change by the organization [24]. This problem is exacerbated by the different types of resources required to support plan-driven versus agile methodologies [4]. For example, agile methodologies generally rely on having strong, informal advice-sharing networks among team members [53]. Team members are often required to collaborate “horizontally” with individuals neither above nor below them in the formal organizational hierarchy. Plan-driven methodologies typically make use of formal communication plans that specify the exact lines of communication (typically “vertical” between supervisor and employee) to be followed for each information type [44]. To overcome this problem, an effective hybrid solution would need to accurately assess, *a priori*, (1) whether each work module would be best fit for a plan-driven versus agile methodology and (2) which human- and technical-based resources should be assigned to each work module.

Literature and Theory Review

WHILE HYBRID APPROACHES EXIST IN A VARIETY OF FORMS IN PRACTICE, little research to date actually prescribes any detailed, formal techniques [4], with one notable exception. Boehm and Turner’s [8] study on the incremental commitment model (ICM) is designed to help practitioners switch between agile and plan-driven methods based on the level of risk in the software development environment. The concept of IT project risk has been well developed [7, 38, 43]. It refers to the potential for unexpected changes or pitfalls in the IT project process that can lead to poor performance. The ICM is designed to allow IT project teams to vary their technique at certain key “decision points” during the process in order to rematch their methodology to changes in risk. Because of this process emergence, the ICM is like an agile methodology except that it can take advantage of plan-driven techniques as the opportunities arise *during* the project.

Our research complements the ICM in that it requires PMs to estimate the level of each subprocess’s risk *before* the design and development phases as well as handling the changing of methodologies during project execution as needed. Up-front risk analysis is performed (1) by using service-dominant logic [56] to modularize the software development workflow and analyze the form of interdependence within each module and (2) with the support of an IT service bus that allows PMs to dynamically assign resources to carry out each work module.

Service-Orientation and Modularization

From a technological perspective, the service-oriented paradigm is simply an “approach for separating concerns” [14]. Service orientation is based on the concept of modularity [2], which refers to “the degree to which a system’s components can be separated and recombined, and . . . both the tightness of coupling between components

and the degree to which the ‘rules’ or the system architecture enable the mixing and matching of components” [42, p. 312]. For example, the division of software into modularized components takes place along natural “break points” in the application where there is minimal interdependence between modules. Parts of the application can be easily changed with marginal interference elsewhere.

As systems become increasingly modular, they also become increasingly complex because of the greater coordination required to manage a larger number of elements [32]. Theory posits [32] that modular systems require (1) an architecture that specifies all modules and their respective functions; (2) interfaces for module interaction, coordination, and communication; and (3) standards for testing the module’s conformity to design rules.

SOA is a “paradigm for organizing and utilizing distributed capabilities that may be under the control of different ownership domains” [33, p. 8]. It provides a uniform means to offer, discover, interact with, and use capabilities to produce desired effects consistent with measurable preconditions and expectations. The project process should be separated into natural break points. Each module should also be described with a standardized interface that allows it to be stored in, and accessed from, an SOA. Each module (now referred to as a “service”) is given a description that includes the contribution it makes to the overall process, potential service providers, necessary preconditions, and measurable postconditions and performance indicators. Finally, a variety of standardized service quality measures can be developed allowing organizations to ensure that the services they use conform to predefined quality metrics [45].

Although SOAs have typically been used to manage service-oriented software applications, their use is beginning to extend to include individuals and groups that can also be viewed as resources that perform services in a business process [6]. These human resources can be described, coordinated, and invoked as services from an SOA using standards for interfacing, description, and discovery [6, 45]. As a result, SOA is defined as a “paradigm” for managing “capabilities” because it can be used to manage any type of distributed capability—whether it is technical (e.g., Web service), human, or organizational [33]. SOAs facilitate the organization and utilization of services that may be distributed across different ownership domains. For example, just as Web services may reside on distributed Web servers within or outside an organization, the human services used to design software modules and write software code may come from internal personnel or external contractors and consultants. It facilitates the use of an SOA that can greatly speed and enhance the ability to organize and reorganize these services with greater agility [13]. We apply these concepts of service orientation, not to the technical architecture of the software being developed, but to the human services that comprise an IT project process.

The Service-Oriented Methodology for Software Development

Comparing SoMSD and SoSE

Methodologies for creating service-oriented systems—often termed “service-oriented software engineering” (SoSE) [51]—already exist in many forms [41, 51]. These

methodologies are optimized for creating software systems based on architectures of Web services or similar technologies. With SoSE, the “service orientation” lies within the technical architecture being developed. Alternatively, the “service orientation” of SoMSD does not refer to the IT project output. Rather, the methodology of SoMSD itself is service oriented² and independent of the IT architecture generated. The SoMSD can be used to create systems that are service oriented, component oriented, object oriented, or based on any other architecture.

The difference between SoSE and SoMSD can also be examined by clearly defining the business problems addressed by each. SoSE techniques are intended to streamline the creation and orchestration of loosely coupled *technical* services (e.g., Web services). By customizing a methodology specific to one technical architecture, SoSE eliminates or enhances certain project processes in order to best meet the needs of a service-oriented technical architecture. As a matter of consequence, SoSE closely resembles agile methodologies [41]. In contrast, SoMSD is intended to be used across a wide range of technical architectures. It is an ideal methodology when the IT project environment is not entirely suited to a plan-driven or an agile methodology alone. As stated above, SoMSD is a hybrid methodology. By basing the systems development process on the creation and orchestration of loosely coupled *human* services, it allows each of the project subprocesses to be independently executed using either agile or plan-driven methodologies. SoMSD is a more general methodology intended for a variety of technical architectures, whereas SoSE is a more focused methodology targeting one architecture.

SoMSD Requirements

The SoMSD has two theory-based underpinnings. First, the common software development processes must be modeled, decomposed, and defined into standardized human services (or human-performed activities). This task is not simply the modularization of project activities. It also involves specifying the inputs and outputs required by the service, the skills and resources (both human and technical) necessary to execute the service, and the measures of, and required levels for, service quality. Figure 2 shows aspects of a service definition.

Similar concepts of human service patterns have also been recommended specifically for service-oriented software architectures [17]. We propose that these service patterns can be generated for any type of IT project architecture. This modularization should be theory driven by dividing those subprocesses that have inherently sequential interdependencies from those that are primarily reciprocal [49]. The contribution of this approach is that it places the focus of modularization on human interaction—the basis of IT project process—rather than technical architecture. A PM may divide the project work based on the technical architecture of the system being developed. In a component-based solution, Team 1 builds technical Component 1; Team 2 builds Component 2, and so on. However, both sequential and reciprocal interdependencies may exist within a single component. Or, a set of related components may all have sequential interdependencies only. As a result, it would be useful to modularize project workflow either more or less granularly than the lines that technical components

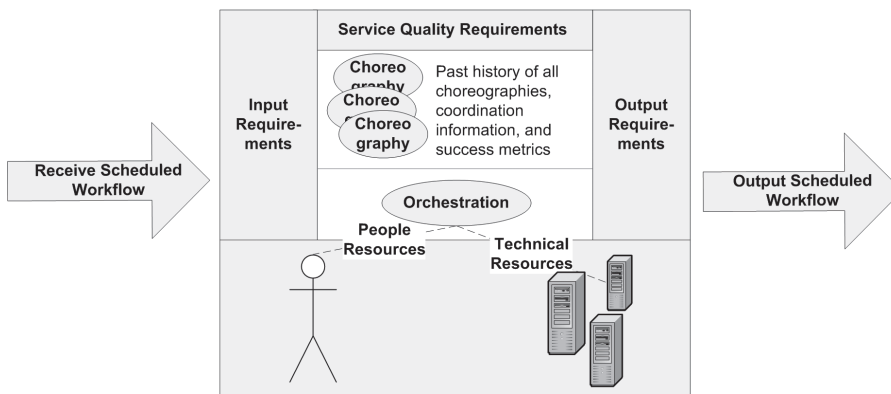


Figure 2. Service Entity Conceptualization

dictate. The PM should identify the natural breakpoints in human interdependencies (whether reciprocal or sequential) and divide project services appropriately. In practice, this task will be completed over time as new projects arise.

Second, based on modular systems theory [32], a type of SOA must be implemented for storing and choreographing these human services. Figure 3 shows a hypothetical IT project with concurrent processes being modularized based on interdependency type,³ stored in a type of SOA of human services, and choreographed into patterns.

The SOA approach accomplishes two basic tasks. First, it consists of a repository for storing service descriptions (both human and technical), the service patterns comprising business processes, the resources used to perform services (e.g., a list of employees, contractors, and consultants and the services they can each provide), and historical patterns of resources allocated to services. Second, the SOA approach used during the SoMSD project execution will help managers dynamically assign resources to services. This SOA approach must be implemented up front in order to store the service definitions that result from the modularization task.

Once these requirements have been met, the SoMSD can be deployed. The basic stages of SoMSD include *service pattern selection* (i.e., service choreography), *resource allocation*, *methodology selection*, and *service execution* (i.e., service orchestration). For organizations migrating to SoMSD from a traditional plan-driven methodology, these steps can be easily mapped into traditional SDLC phases (see Figure 4). To help describe the SoMSD process, the following subsection provides a hypothetical example.

SoMSD Example

First, the PM searches the SOA service pattern repository for a service pattern that best represents the services necessary for the new project.⁴ If no appropriate pattern exists, the PM will create a new pattern. Figure 5 represents a hypothetical service pattern the PM might select as the closest match. The boxes labeled “Service A,” “Service B,”



“Service C,” and “Service D” represent less-granular or high-level services. Each of these services is composed of its own pattern of subservices ($A_a, A_b, A_c, B_a, B_b, B_{b2}, B_{b3}, B_c, C_a, C_b, C_c, D_a, D_b$). Some subservices are composed of their own pattern of sub-subservices (B_{bb}, B_{bc}). Each level (service, subservice, or sub-subservice) possesses the same generic properties (e.g., inputs, outputs, quality) and differs only in terms of granularity.

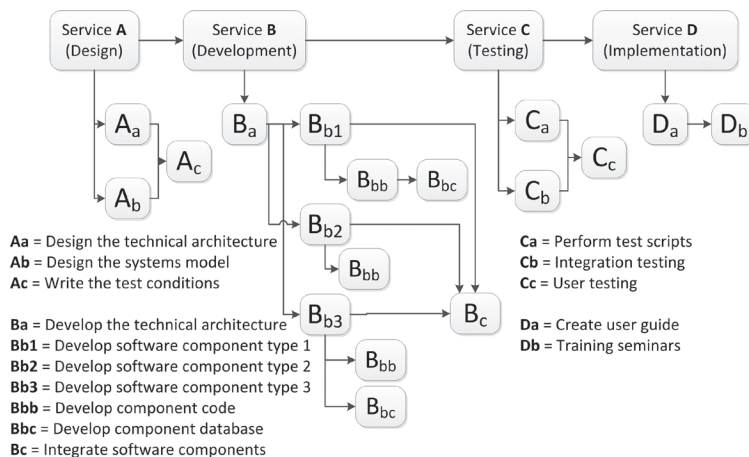


Figure 5. Service Pattern Example

The subservice pattern that comprises the “development” service (Service B) demonstrates how each activity may have multiple potential service patterns. In the example in Figure 5, three components are produced in the development process (characterized by subservices B_{b1} , B_{b2} , and B_{b3}), each with a unique pattern of sub-subservices. Subservice B_{b1} is performed by developing the component code first followed by the component database. Subservice B_{b2} does not need a database component. Subservice B_{b3} can be performed by developing both the code and database simultaneously.⁵ Once a matching (or closely matching) service pattern is found, the PM performs a feasibility estimate of moving forward with the project. This feasibility analysis can be based on the cumulative cost estimates from each of the service descriptions.

If the project is financially feasible, the PM can then select and allocate resources. If the selected service pattern has been used previously, the PM can search the repository for resource allocation patterns linked to that service pattern. Resource allocation patterns are a historical record of the actual patterns of software, hardware, and human services assembled to accomplish prior projects. If no resource allocation patterns are available, the SOA mapping engine helps the PM to identify appropriate and available resources for each service.

Figure 6 shows a hypothetical resource allocation pattern for the *Development* service only (Service B) from Figure 5. The resource numbers represent unique identifiers. The example demonstrates how some people may be assigned to multiple services (e.g., Person 6 works on both sub-subservise B_{bc} and subservice B_c). Each service, subservice, or sub-subservise has a set of patterns for how it might be accomplished depending on which resources are available.

One of the most important benefits of the SoMSD is derived from this process of resource allocation. This is because “people resources” are stored in addition to technical resources. In other words, each potential project team member (including employees, contractors, and outsourcing partners) should be stored along with a list

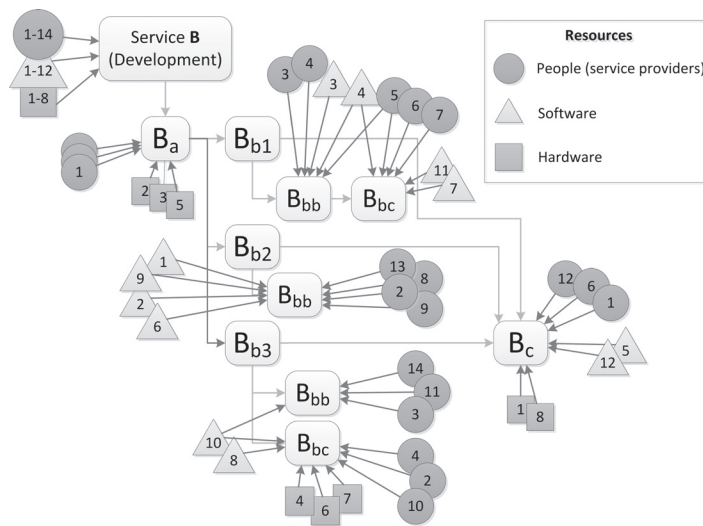


Figure 6. Resource Allocation Example

of standardized services that they are capable of performing. As services are defined in greater granularity and detail, the people resources can be better utilized across all project services.

In addition, the SOA repository should contain schedule and availability information so that it can dynamically assign people to services. The task of resource allocation can be streamlined, and there will be a reduced error rate and less uncertainty. A PM may begin by selecting a particular resource allocation pattern only to discover that some individuals have scheduling conflicts with other project assignments. The SOA can search the repository of service providers and suggest alternatives. Some of the service providers suggested by the SOA may be external contractors or consultants if internal personnel are not available, as represented in Figure 6. Thus, the SOA can facilitate more targeted outsourcing [13]—particularly if the outsourcing partners are given access to the SOA repository of service definitions so that they can build their own service offerings based on the needs of their clients employing the SoMSD.

In Figure 7, subservice B_{b3} is inverted to represent an outsourced service. Notice that there are no resources assigned to that service because the SoMSD organization does need to know how the outsourced provider is performing the service—only that it produces the necessary outputs.⁶ When service provider availability is established, the SOA can schedule the time needed from each employee in its master schedule.

Each service provider performs his or her “subproject” using the most appropriate methodology, whether agile or plan driven. For those services being performed in-house, the PM may choose to mandate each service provider’s methodology based on the PM’s initial assessment of interdependency or allow the teams to self-select. The new service and resource allocation patterns that are created for the project are recorded in the SOA repository.

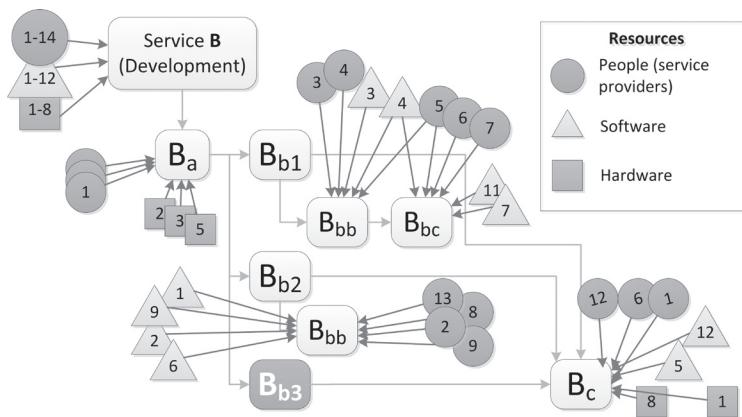


Figure 7. Service Outsourcing Example

How Does SoMSD Meet Solution Objectives?

Regarding Objective 1, we have argued that modularization should separate those subprocesses with sequential versus reciprocal interdependencies based on interdependence theory [49]. The SoMSD requires that each modularized subprocess be modeled using a standardized service definition. These service definitions are stored in a repository that allows shared access to individuals and groups within the organization as well as external contractors. As seen in Figure 2, each service definition includes (at a minimum) the input/output requirements, the human/technical resources required to complete it, and quality requirements. Just as important, each service provider records standardized historical information such as the levels and types of coordination required, the methodology used, and success ratings. While the SOA will record which methodologies were used to perform the service in the past, the service definition and quality requirements do not specify that an agile versus plan-driven technique be used. The methodology is independent of the service being performed as long as the appropriate inputs are accepted and outputs are produced with an acceptable level of quality.

The services that can be decomposed to a more granular level of subservices are those with low risk. These can be executed using a plan-based methodology. By contrast, services that cannot be decomposed to a relatively granular level have high risk and will need to be executed with an agile methodology. This could result from the requirements having not been adequately defined or because there is no past history of completing those services, potentially leading to the use of agile methodologies for performing these services. The process of creating service definitions for each task becomes a natural indicator of which methodology will be most appropriate for each service. This task makes it easier to identify smaller portions to outsource to external contractors.

Concerning Objective 2, the SoMSD is supported by the use of an SOA, or at the least, an ontology editor, which is a shared repository of services, service patterns,

and resources [55]. During the planning and analysis phases of a project life cycle, this ontology allows PMs and analysts to draw from prior project knowledge and experience. In practice, the ontology also stores and makes accessible the records of actual past implementations of each service, the historical quality and performance of those implementations, and the typical patterns in which each service has been used. It enhances the knowledge base that future projects can draw from. By reusing past project knowledge more extensively, the risks and uncertainty that result from poor planning and analysis can be reduced in future SoMSD projects. This will reduce the time required to plan, analyze, and source new projects. Even if new projects are very different from any past project, it is still possible to reuse the more granular patterns of past projects if the services have been modularized to sufficient detail. The SOA, or ontology editor, used in SoMSD provides the PM with a more complete knowledge management system than is often used in practice.

Results of an SoMSD Implementation

THE SoMSD WAS ORIGINALLY INVESTIGATED as part of an action research project with the business intelligence unit of a Fortune 100 financial services organization (termed “FSC” for nondisclosure purposes) from early 2005 until late 2007. FSC’s software development unit deemed agile methodologies to be too risky given their high rate of employee turnover and inadequate knowledge sharing across team boundaries. They initiated a project with us to develop an SoMSD. During this project, qualitative research methods were used to understand the effects of SoMSD adoption. Although FSC’s IT project time and cost estimates significantly improved as a result of SoMSD adoption, a rich set of quantitative measures could not be released because of nondisclosure restrictions. Additional quantitative data were collected from another source. Because SoMSD is new to most organizations, theory and research assessing the effects are obviously limited, making the case study methodology most appropriate for testing the effectiveness of SoMSD [5].

A case study was initiated with a large electrical power company (termed “EPC” for nondisclosure purposes). EPC is highly recognized industry leader and trend-setter in advanced technologies and processes for electrical power companies. The EPC case study took place over two years from November 2007 to November 2009. Their initial software development methodology was plan driven and based on the traditional waterfall approach. Approximately one year prior to the study (November 2006), EPC began an initiative to model the activities of their software development process and break them down into the core activities in a similar manner to the service patterns used in SoMSD. They were not yet reusing these models or any resource allocation patterns, but a more standardized storage and reuse had been discussed for future projects before this study began. EPC’s IT unit spanned multiple campuses and domains. The particular subunit examined within EPC was responsible for maintaining their homegrown customer relationship management system. This unit had 60 full-time employees and 100 in-house contractors at any time. Demographically, on average, this unit was 60 percent male, 34 years old, 68 percent Caucasian, 16 percent Hispanic,

and 9 percent African American. Four additional employees existed as part of a business process analysis (BPA) team whose primary responsibility was to develop what they termed to be “enhanced project process models,” which were primitive service patterns and resource allocation patterns.

The creation of the BPA group began because many projects exceeded budget and schedule estimates. The unit director and senior management both agreed that one of their greatest problems was the poor business process modeling that took place during the planning and analysis stages. The BPA group was formed and operated directly under the unit director. Shortly after the BPA group was formed, they decided to model not only the AS-IS and TO-BE⁷ models of the business processes affected by the projects but also the software development process itself. This decision is the first step in the SoMSD adoption of developing service patterns.⁸ It was soon after this point when the formal SoMSD adoption began.

Data Collection

To understand and measure the effect of this SoMSD adoption, a variety of objective data and manager perceptions were gathered from 11 separate projects. These projects were selected to provide a sample from before SoMSD adoption and through various stages of SoMSD maturity. Each project involved improvements to EPC’s customer relationship management system. Each PM was required to complete a standardized monthly report that included the current progress of each of the six project stages (planning, analysis, design, implementation, testing, and support), actual completion dates for each stage and total expenditures to date by stage, projected completion dates and costs, expected project risks and the effects of those risks, and formalized changes to project scope. This allowed the unit director to know how far over or under budget the project was at that point. In total, 124 monthly project reports across the 11 projects were collected in order to investigate how project schedule and cost estimates changed over time.

In addition to project monthly reports, other formal project documentation were collected, such as communication plans, documentation about the key technology resources used for each project, organization charts, and the governance and authorship of process models. Also, the actual business and project process models were collected. Finally, 45 structured and unstructured interviews were performed with the PMs of each project, the unit director, senior management representatives, the chief external consultant for EPC management, administrative staff, and members of the BPA group. These interviews gave insight into which variables would be most relevant in capturing the effect of SoMSD adoption. A formal survey was administered to the PMs, the unit director, senior management representatives, and subject matter experts to collect additional perceptual data to compare management perceptions to the objective project data.

Measures

In general, research studies on IT project success have focused on independent variables concerning the various forms of risk and coordination mechanisms intended to manage risk (e.g., [3, 16, 30, 38]). Therefore, in addition to collecting project per-

formance data, we also collected measures of (1) formal coordination mechanisms, (2) informal coordination, (3) pre-design stage risk factors, and (4) post-design stage risk factors.

The measures of coordination mechanisms are based on those used in prior research and field studies in particular [30, 35]. The formal coordination mechanisms measured included (1) the degree of SoMSD adoption, (2) the group structure, (3) the governance over business and project process modeling, (4) the technology used to create and store process models, and (5) the coordination between the IT group and the BPA group in creating the models. SoMSD adoption is measured as the extent to which each project implemented the SoMSD principles relative to the other projects. For example, each project's service patterns varied in terms of level of completeness and granularity—and the percent of each project's models that were reused from prior projects. Group structure was determined by examining formal organization charts and conducting interviews with the unit director. Groups with a *divisional* structure were those that included all of the diverse roles needed to manage an entire project while those with a *functional* structure included a single type of role used across many projects. Governance over business and project process modeling refers to whether the IT group or the new BPA group had responsibility for the completeness and quality of models. The technology used to create the models was either Microsoft Visio or Metastorm ProVision®—a powerful, object-based tool that offers the ability to reuse objects created from one model to another. The coordination between the IT group and the BPA group was measured in terms of the percent each group contributed to the final project models. Determining the percent contributed by each side was estimated based on examining the recorded attendance at meetings during the model development phases and the recorded authorship of electronic model documents. For example, while the BPA group may have governance over the models, they may ask the IT group to produce certain portions and attend meetings to collaborate on the final models.

Informal coordination was measured as (1) whether or not the PM developed a communication plan, (2) the level of detail of the communication plan, and (3) the perception regarding how effective and necessary the plan was to completing the project.

Pre-design stage risk measures include the PM's and unit director's perception of how realistic project schedules and budgets were, the cost of scope changes approved in the pre-design stage (normalized by initial project budget), and the percent over/under budget the project was for the pre-design stages (planning and analysis). Post-design stage risk measures include the PM's perception of the amount of fluctuation in requirements,⁹ the percent over/under budget, and the percent over/under schedule—all during the post-design stages.

Overall project performance was measured (1) objectively by the percentage over/under budget and schedule for the entire project and (2) subjectively by the combined perception (of the PM, unit director, and senior management representative) of how well each project delivered its key features and was overall successful.

Each of the perception-based variables listed above were measured based on existing scales [3, 16, 38, 43]. Of the subjective measures, only project success was measured using reflective items and exhibited strong reliability ($\alpha = 0.82$). Each of the other variables either used single-item measures or were based on formative

constructs—meaning the indicators could vary independently of each other [28]. As a result, traditional tests of reflective measurement validity do not apply [36].¹⁰ Table 1 summarizes the measures.

Results

Table 2 displays the 11 projects examined during this study and the adoption criteria for each. Projects 1 through 5 had no project patterns modeled at all. However, the new methodology for project process modeling and the preparation of reusable patterns began during Project 5. An increasing number of project service patterns were developed in Projects 7, 8, and 9. According to the unit director, these projects represent a more advanced SoMSD adoption than Projects 5 and 6, which they called the “learning stage.” In this stage, the BPA group member who initiated the methodology trained others in this group on the technique and presented to key members of EPC to introduce the new methodology. Projects 10 and 11 had the most project processes modeled and were the first two to actually make use of reused process models from prior projects. Therefore, EPC considered these projects in the “maturing stage.”

Project Coordination

Table 3 summarizes the changes to formal coordination mechanisms as the SoMSD was adopted. During the preadoption period, the IT group controlled the modeling that took place. During this time, EPC used a divisional group structure that is based on a formal hierarchy and separated roles. This is typical of teams using plan-based methodologies [37].

After the formal creation of the BPA group, the IT group kept the modeling responsibility for only one more project (Project 5) before formal governance was handed over to the BPA group. In Project 6, the IT group once more created all of the business process models, but the BPA group built on these models to create a more detailed and modular set of models. During this time, EPC also began a transition to a cross-functional structure by breaking apart project groups into functional groups and then temporarily assigning people to temporary project teams. The unit director felt that as the project scope and responsibility became clearer, they could benefit by placing those performing similar roles into the same groups to create a “job shop” environment.

During Projects 7, 8, and 9, the new BPA group made greater strides in completing the project modeling before the execution stages began. According to the BPA team, establishing a formal modeling methodology to follow made it easier to finalize models and gain approval from management. In addition, they were careful to include the IT group’s help and opinion into the models they created. Also during this time, the BPA group developed standardized project models for the first two project stages and adopted the object-based modeling tool ProVision.

Table 4 summarizes the changes in informal coordination across projects. Although there was no expected direct effect of the SoMSD on the use or effectiveness of communication plans, the plans themselves began to improve as the SoMSD was adopted.

Table 1. Summary of Variables Captured at EPC

Variables	Type	Measure/item	Range	References
SoMSD adoption				
Patterns modeled	Observed	PM and BPA representative	0–100%	[52]
Patterns reused	Observed	reviewed all project models and jointly estimated the percentages	0–100%	[53]
Formal coordination mechanisms				
Modeling ownership	Observed	Based on a review of project records	IT versus BPA	N/A
Modeling by IT group	Observed	Based on a review of model authorship	0–100%	[29]
Modeling by BPA group	Observed	Based on a review of model authorship	0–100%	[54]
Modeling in Stages 1–2	Observed	Based on a review of model metadata	0–100%	[16]
Modeling in Stages 3–5	Observed	Based on a review of model metadata	0–100%	[9]
Technology used	Observed	Based on electronic document types	Visio versus ProVision	[61]
Group structure	Observed	Based on formal organization charts	Divisional versus functional	[59, 61]
Informal coordination mechanisms				
Communication plan used	Observed	Based on project records	Yes/no	N/A
Level of plan detail	Observed	Relative based on all communication plans	High/medium/low	N/A
Plan quality	Latent	This project had realistic and explicit communication plans	1 to 7 ^a	[38]
Necessity of informal coordination	Latent	This project required frequent informal coordination	1 to 7 ^a	[38, 52, 61]
Pre-design stage risk factors				
Realistic estimates	Latent-formative	This project had realistic resource estimates	1 to 7 ^a	[29]
		This project had realistic scope estimates	1 to 7 ^a	[54]
		This project had realistic schedule estimates	1 to 7 ^a	[54]

(continues)

Table 1. Continued

Variables	Type	Measure/item	Range	References
Scope changes	Observed	Approved scope changes based on recorded documentation ^b	1 to 52	[53]
Normalized scope changes	Observed	Number of approved scope changes/total project budget	0 to 0.000021	N/A
Over/under budget	Observed	Based on project records	−47.5% to 22.5%	[9]
Post-design stage risk factors				
Requirements fluctuation	Latent	Requirements fluctuated during the construction, testing, and implementation stages	1–7 ^a	[53]
Over/under budget	Observed	Based on project records	−33.8% to 249.7%	[16]
Over/under schedule	Observed	Based on project records	−22.8% to 442.7%	[9]
Overall project performance				
Over/under budget		Based on project records	−52.1% to 622.7%	[9]
Over/under schedule		Based on project records	−27.3% to 424.8%	[9]
Overall perceived success	Latent-reflective	This project delivers all features and functionality	1 to 7 ^a	[50]
		This project meets key objectives and business expectations	1 to 7 ^a	[56]
		This project overall is very successful	1 to 7 ^a	[56]

Notes: N/A = not applicable. The references refer to either the source of the latent items or the basis for the observed measures. ^a Latent items based on a Likert scale anchored by 1 = “strongly disagree” and 7 = “strongly agree.” These items were answered by each project’s PM and the unit director. ^b Only those designated by EPC as “medium” to “large” were included.

It may simply be that the new focus on “attention to detail” that came with the SoMSD adoption influenced the effort the PMs placed on these communication plans. Regardless, the perceived necessity of informal communication decreased as SoMSD was adopted. This supports the notion that increased formal coordination in the pre-design stage can decrease the need for informal coordination in the post-design stage [16]. However, the need for informal coordination will never be eliminated because some scope changes are unrelated to the quality of the initial plans.

Table 2. Overview of the Progression of EPC's SoMSD Adoption by Project

Adoption stage	Project	Percent of project service patterns modeled	Percentage of patterns reused
Before adoption	1	0/100*	0
	2	0	0
	3	0	0
	4	0	0
Early stage	5	20	0
	6	50	0
Learning stage	7	90	0
	8	70	0
	9	80	0
Maturing stage	10	100	20
	11	100	50

* Project 1 initially was not modeled. However, because the project went far over schedule, it was "restarted" after the testing phases revealed some major flaws. The 100 percent modeling was not completed until the second attempt.

Table 5 summarizes the changes in pre- and post-design stage risk factors across projects. During the pre-design stage, the PM's perception of how realistic the schedule, budget, and scope estimates were greatly increased from an average of 3.4 (1 = "most unrealistic," 7 = "most realistic") before SoMSD adoption to 6.1 by the time the SoMSD became mature. Similarly, the number of scope changes normalized by project size decreased dramatically from an average of 0.00026 before SoMSD adoption to less than 0.000001 at SoMSD maturity. Perhaps most telling, the percent that the project was over/under budget in the pre-design stages decreased from an average of 11.2 percent *over* budget before SoMSD adoption to 5.9 percent *under* budget at SoMSD maturity. It is important to note that this change in over/under budget is a very conservative estimate because the percentages pre-SoMSD were significantly altered each month as the project scope increased. If the initial project budgets were used, then the pre-SoMSD projects would have been more than 103 percent over budget on average. Also, the unit director quickly learned as the SoMSD matured that she could stop inflating the budgets because of uncertainty. In particular, she no longer had to "add 25 percent to 40 percent to the PM's estimate" after SoMSD maturity. In summary, the improvement in EPC's ability to meet their budget pre-design stage was much larger than the 17.1 percent swing in Table 5 indicates.

Post-design stage risk factors experienced similar improvements. The perception of requirements fluctuation decreased from 6.1 before SoMSD to 2.85 at SoMSD maturity, the percent over/under budget decreased from 28.3 percent over budget to 1.6 percent over budget, and the percent over/under schedule decreased from 213.3 percent *past* schedule to 17.4 percent *ahead* of schedule.

The change in overall project performance was similarly drastic (see Table 6). The percent over/under budget for the entire project (based on the initial project estimates)

Table 3. Changes to Formal Coordination Mechanisms by Project

Project	Governance	Coordination		Technology	Structure	
	Process modeling ownership	Percent of modeling contributed by IT side	Percent of modeling contributed by BP analysts	Percent of modeling in stages 1,2/3,4,5	Service pattern editor	Group structure
1	IT/BPA	90/0	10/100	0/100	N/A/ProVision	Divisional
2	IT	80	20	90/10 ^a	N/A	Divisional
3	IT	50	50	50/50	N/A	Divisional
4	IT	100	0	10/90	N/A	Divisional
5	IT	70	30	0/100	Visio	Cross-functional
6	BPA	100	100	100/100 ^b	Visio	Cross-functional
7	BPA	50	50	80/20	ProVision	Cross-functional
8	BPA	50	50	0/100 ^c	ProVision	Cross-functional
9	BPA	20	80	100/0	ProVision	Cross-functional
10	BPA	50	50	100/0	ProVision	Cross-functional
11	BPA	50	50	100/0	ProVision	Cross-functional

Notes: N/A = not applicable. The modeling referred to above is the normal business process model that takes place in most traditional, plan-driven projects—not the software development process modeling that is part of the SoMSD. ^a Project 2 had what the unit director considered to be the best PM she’s ever had. As a result, several of this PM’s coordination and risk factors appear much better than the other pre-SoMSD projects. ^b The first models created for Project 6 were not adequate and had to be 100 percent recreated by the BPA group. ^c The modeling in Project 8 was atypically pushed to the later stages because of major changes in stakeholders.

Table 4. Changes to Informal Coordination by Project

Project	Communication plan?	Detail	Effectiveness of informal coordination	Necessity of informal coordination
1	No/yes ^a	N/A/high	1	7
2	Yes	High ^b	5 ^b	5
3	No	N/A	2	6
4	Yes	Low	4	6
5	No	N/A	2	5
6	Yes	Low	5	5
7	Yes	Medium	5	5
8	Yes	High	5	4
9	Yes	High	7	3
10	Yes	High	6	4
11	Yes	High	7	3

Notes: N/A = not applicable. ^a Recall that Project 1 had to be restarted because of the failure of the first attempt. The cell indicates that a communication plan did not exist for the first attempt while it did for the second. ^b High rating due to the quality of the PM.

decreased from 281.4 percent *over* budget before SoMSD adoption to 5.5 percent *under* budget at maturity. The percent over/under schedule decreased from 206.4 percent *past* schedule before SoMSD adoption to 3.6 percent *ahead* of schedule at maturity. According to survey respondents, the perception of overall project success increased from 5.3 (1 = “lowest success,” 7 = “highest success”) before SoMSD to 6.8 at maturity. However, their perception regarding each project’s ability to deliver all key features and objectives increased from 4.4 (1 = “lowest delivery,” 7 = “highest delivery”) before SoMSD to 6.8 at maturity.

Discussion

THE SoMSD WAS ASSOCIATED WITH CHANGES IN FORMAL AND INFORMAL COORDINATION. These changes were correlated with decreases in project risk and uncertainty. Overall project success improved dramatically in terms of EPC’s ability to meet budget and schedule estimates as well as management perceptions. The process by which EPC transformed during their adoption of the SoMSD has several interesting implications for practice and research on SoMSD.

Implications for Research

An underlying proposition in this study of systems development methodology is that theory on service orientation and modularity can inform the creation of hybrid methodologies such as SoMSD. Through the proper use of SOAs for managing service complexities, SoMSD will improve an organization’s IT project performance if

Table 5. Changes in Risk Factors by Project

Project	Pre-design stage risk factors			Post-design stage risk factors			
	Realistic estimates?	Number of scope changes	Changes normalized by budget	Percent over/under cost	Requirements fluctuation (1 = lowest; 7 = highest)	Percent over/under cost	Percent over/under schedule
1	2.1	40	0.000021	12.2	7	-3.6	123.6
2	2.6	52	0.000019	0.4*	5	9.0	1.2
3	4.1	44	0.000054	9.7	6.3	1.8	285.8
4	4.7	16	0.000010	22.5	6	99.8	442.7
5	5.1	3	0.000002	9.8	6	46.5	0.9
6	5.8	8	0.000007	-47.5	4	249.7	40.5
7	4.6*	6	0.000004	0.3	4	13.8	-22.8
8	5.7	7	0.000001	-2.8	6.5*	-33.8	-3.6
9	5.8	2	0.000001	-3.6	3	-5.4	-55.4
10	6.1	1	0.000000	-12.9	2.7	-3.4	-13.1
11	6.1	1	0.000001	1.1	3	6.6	-21.7

* Positive scores due to the quality of the PM.

* Positive scores due to the quality of the PM.

Table 6. Overall Project Performance

Project	Percent over/under cost	Percent over/under schedule	Delivers all key features and objectives?	Overall perceived success
1	332.9	95.2	3	5
2	153.0	99.6	5.7	6.5
3	622.7	205.8	4.7	5.7
4	17.1	424.8	4	4
5	16.2	34.2	5	6.3
6	55.1	29.4	6.7	6.7
7	-52.1	13.9	5	5.5
8	-14.0	-27.3	2.7	3.3*
9	-15.9	3.9	5.7	6.3
10	-1.7	5.6	7.0	7.0
11	-9.3	-12.7	6.5	6.5

* While Project 8 exceeded budget and schedule estimates, its scope was heavily debated by politically motivated stakeholders, resulting in lower perceptions about its success.

the task environment is characterized by a mix of reciprocal- and sequential-based subprocesses. This is because the SoMSD represents a better fit between coordination strategy and interdependency than methodologies that are primarily agile or plan driven in nature.

Proposition 1: In task environments characterized by both reciprocal and sequential interdependencies, hybrid methodologies, such as SoMSD, perform better than methodologies that are primarily agile or plan driven.

Although the results of EPC's SoMSD adoption clearly show changes in their coordination mechanisms and project risks, these changes did not follow a consistent chronological pattern. In other words, it cannot be concluded that project risk always determined coordination mechanisms or that coordination always determined risk. Rather, the management decisions following SoMSD adoption should be interpreted as an effort to achieve optimal fit between project risk and coordination strategies. As a result, we draw from information processing (IP) theory [15] to frame the events at EPC and to establish propositions.

IP is a contingency theory based on achieving fit [15]. It posits that an organization's performance is heightened when its IP need is congruent with its IP capabilities. It is not optimal to have either too much or too little capability. IP needs are the result of interdependency [54]. For example, reciprocal interdependencies create greater IP needs than sequential interdependencies [49]. Therefore, managers must develop strategies for increasing IP capabilities when too much reciprocal interdependency exists. Three strategies have been established in theory: (1) acquire improved technology [12], (2) change group structure to collocate reciprocal interdependencies [49], and (3) improve horizontal communication [54].

Each strategy was adopted at EPC, but not entirely as might be predicted. Before EPC's SoMSD adoption, their IP need was high. This is apparent from the high levels of risk in Projects 1–4. Scope changes increased during the first three projects, and the pre-design stage schedule and budgetary overages peaked in Project 4 (see Table 5). As a result of their high level of risk, EPC employed the second strategy above and formed the BPA group, which collocated employees with the most reciprocal interdependencies. Previous to this BPA group, the IT side had to collaborate across formal hierarchies with the business side in order to create business process models. Conversely, the new BPA group had both formal and informal communication relationships with employees in the IT and business sides, which reduced horizontal communication cost.

Soon after the BPA group began creating models, they needed a way to store and share the models for reuse. Although SoMSD does not necessarily require the formation of a BPA group, it does mandate that an SOA technology be implemented. The SOA's repository for storing service patterns is a form of ontology [55] that provides shared access to service definitions and service patterns both within the organization and with external partners. The SOA greatly increases the BPA team's ability to process new project plans and resource estimates. Therefore, EPC employed the first strategy above by adopting a new technology for IP.

As EPC made progress in documenting service definitions and patterns, it became clear that their existing project team-based structure was no longer necessary. For example, employees were originally grouped in a divisional structure—meaning that a single project team contained all the skills necessary to complete an entire project (see Figure 8). As a result, all of the project interdependencies were contained within a single group. But, because SoMSD adoption requires modularizing project tasks around those with reciprocal versus sequential interdependencies, it minimizes the interdependencies among different project team roles. Therefore, the strongest reciprocal interdependencies will exist not between different roles (i.e., service providers), but between those who are working together on the same service or role. As a result, based on interdependence theory [49], organizations will group employees around similar functions. In a project-based environment, this translates to the use of *cross-functional* teams where a PM borrows, or purchases, the use of service providers from various functional teams to create a temporary project team. The use of cross-functional teams was not a requirement of the SoMSD, but nonetheless a theoretically motivated outcome that was also observed at EPC.

Proposition 2: SoMSD adoption leads to, and is best supported by, the use of cross-functional group structures.

Consistent with IP theory, all three strategies for increasing IP capabilities were observed at EPC. However, informal communication adjusted to fit not only with the IP needs but also with the other IP capabilities. This is because it is not always optimal to increase all types of IP capabilities if they are not needed [54]. Consequently, if different capabilities can act as substitutes for each other, then adding more of one type of capability may allow an organization to reduce another. Such was the case at EPC. Recall the high level of IP need in Projects 1–4 because of the high project

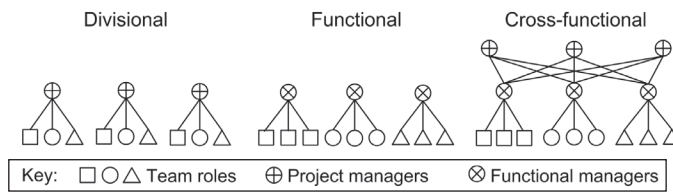


Figure 8. Group Coordination Structures

Source: Adapted from [34].

risks. Because no changes were made to the technology or group structures during those projects, the level of horizontal communication necessarily increased in order to compensate. This is apparent from the high need for informal coordination stated by the PMs from Projects 1–4 (see Table 4). Interestingly, after the new BPA group was formed and cross-functional teams were adopted, the perceived need for informal coordination slowly began to decrease in Project 5 and leveled out during Projects 8–11. Therefore, as formal coordination mechanisms are improved, informal coordination may naturally decrease in the long run until IP needs and capabilities balance.

Proposition 3: SoMSD adoption leads to a relative decrease in informal coordination and an increase in formal coordination.

Proposition 3 has important implications for research and theory on the role of advice networks [47]. Social network research has typically viewed advice-sharing relationships as a positive indicator of individual and group performance [47]. However, EPC's results implicate that informal social network connectivity—in the form of advice-sharing relationships—in software development environments is actually an indication of the degree of project risk not managed through formal means like group structures and technology. In other words, a high degree of informal knowledge sharing may be a red flag that a project is facing too much interdependence and risk—particularly if the project team is employing a plan-driven methodology that should not require so much horizontal communication. As a result, the level of informal knowledge sharing in various subprocesses within the IT project is also a key indicator of reciprocal interdependencies. PMs who are decomposing their project based on interdependencies adoption should analyze the informal knowledge sharing and use it as an indicator of the type of interdependency in each subprocess.

In summary, EPC's results demonstrate that SoMSD adoption will mandate that certain new IP capabilities be adopted (e.g., SOA technology) and lead to natural adjustments in other IP capabilities as well (e.g., cross-functional groups and reduced informal coordination).

Practical Implications and Limitations

While the SoMSD appears to have significant benefits, it is important to discuss some of the practical implications and limitations of SoMSD adoption. Foremost, although

EPC made no other major changes to their supporting IT infrastructure or personnel, we cannot rule out the possibility of other factors in determining their drastic improvements in IT project on-time and on-budget percentages. There are a variety of factors that could have influenced their recent success. For example, simply by implementing a new methodology in general and gaining the buy-in from all IT project team members, the PMs may have generated greater effort and interest, which could account for much of the success. The most significant confounding influence might be EPC's improvement in the business process modeling of the AS-IS and TO-BE models. This traditional task of plan-driven methodologies was not well performed before the SoMSD implementation and was definitely improved when EPC undertook the task of modeling their software development processes as well. Indeed, many of the changes concerning formal and informal coordination mechanisms listed in Tables 3 and 4 are endogenously or exogenously related to the SoMSD implementation. However, this issue of causality is one of the trade-offs made when adopting field research. Experimental control is exchanged for a more realistic test environment. Therefore, a promising area for future research would be to examine the SoMSD principles in a relatively more controlled experimental environment and discover how the SoMSD will influence, and be influenced by, other project factors.

It is also important to note that the SoMSD was tested in an organization that was currently using a plan-driven methodology. The steps of service pattern selection and resource allocation in SoMSD resembles the stages of a plan-driven methodology. Therefore, it may be easier in practice for an organization to switch to SoMSD if it is currently using a plan-driven methodology. This type of switch may require fewer changes to the existing culture and less "mindshift" than switching from an agile methodology [26].

Although we posited that the SoMSD will reduce the interdependencies in a project process, in practice, the SoMSD can also create new complexities and interdependencies. As discussed earlier, as systems become more modular, they also become more complex because of the greater number of elements that must be managed [2]. Software development is performed not by a single integrated project team, but a set of loosely coupled interdependent service providers. As a result, some unexpected problems can occur. For example, if a particular service or service provider is reused too excessively, bottlenecks may be created that can slow down all projects [22]. PMs must schedule the workflow properly so as to help manage the dependencies of the service providers who become bottlenecks. Future research should examine the job shop literature [39] to understand how to best schedule and organize SoMSD workflow.

In fact, the "job shop" environment is one of the unique consequences of SoMSD. Because a list of potential service providers is stored in the SOA, PMs have the ability to choose the service provider they deem "best" for the job. As a result, a market-like competition emerges where team members' performance is evaluated by a "mini-market." High performers are selected more often than low performers. As a result, team members become incentivized to expose all of their capabilities and perform at a maximum level. It also promotes continual education so that one's list of service capabilities continues to grow.

As with agile methodologies [57], the SoMSD may require cultural changes. If power changes hands when the structure is realigned, there may be pushback and resistance. Another issue that becomes exacerbated by the SoMSD is that many of the interdependencies are between service providers who cross organizational boundaries. If the SoMSD enables greater outsourcing, then PMs lose a degree of control over how the project is performed [31]. They become more vulnerable if their outsourcing partners fail to meet service requirements.

Perhaps the primary drawback or barrier to adoption of the SoMSD is that it requires a significant amount of time and effort up front to build the repository of service patterns, resource patterns, and service provider schedules and capabilities. Business and systems analysts must take the time to (1) identify the relevant services that are needed during typical software projects, (2) create standardized service definitions, and (3) create an initial set of service patterns, subpatterns, and so on, and resource allocation patterns. In addition, standards must be developed so that the process of creating and storing new patterns may be accomplished in a way that will allow a shared understanding of the patterns by all PMs. The research into ontologies (the design of shared understanding in a given domain) would be particularly useful toward this end [55]. In addition, recent research has emerged that can help establish human service pattern standards [17].

PMs and analysts may also find the task of modularizing project processes into services somewhat difficult in the early stages. Before SoMSD adoption at EPC, the PMs typically modularized tasks based on the technical architecture. For example, Task 1 consisted of creating Technical Component 1, Task 2 was Technical Component 2, and so forth. However, after adopting SoMSD, the PMs began to focus on differences in human interdependency as a means of modularization. They had to be very observant of human behavior and seek additional feedback from developers. As a result, the appropriate modularization was often learned simply by trial and error. However, EPC also performed social network analyses [47] that helped identify bottlenecks and decipher between sequential and reciprocal interdependencies. Future research should generate techniques for identifying workflow interdependencies.

Also challenging is the task of storing the capabilities, schedules, and capacities of each potential service provider whether internal or external. Careful attention must be paid to the prioritization of services if team members can be assigned to multiple services at a time. Although EPC began their SoMSD adoption without completing these tasks, the speed and accuracy of their resource allocation patterns were greatly improved as they made progress toward this end. New tools can facilitate this task, such as Microsoft SharePoint 2013, which allows individual profiles to be recorded for each employee, including business and technical skills and support for shared calendars. Future research could benefit SoMSD by developing theory-based optimization models for resource allocation patterns.

In this paper, we have argued that the submethodology decision for each service (agile versus plan driven) should be made based on the type of interdependency that dominates each service. However, other factors also influence this decision such as service provider team size, team member tenure and skills, and more [4]. Existing

research that accounts for these other factors will help guide PMs in this submethodology decision making [4, 8]. As SoMSD techniques become more prevalent, future research should expand this research for the SoMSD context.

Lastly, PMs and unit directors should be aware of how their existing coordination mechanisms may need to be adapted for successful SoMSD implementation. It was identified above that managers may need to adopt cross-functional teams. Also, although the SoMSD mandates the use of an SOA, other types of technology may need to be adopted as well. However, with a focus on shared access to patterns and resources, the SoMSD creates unique privacy concerns from storing the capabilities of employees and sharing service definitions with external partners. Future research should examine how these concerns should be managed.

Conclusion

THIS PAPER CONTRIBUTES A SOFTWARE DEVELOPMENT METHODOLOGY based on existing concepts of modularity [1, 2], service orientation [13], and human service patterns [17] to provide a theoretically based solution for software development units that do not fit the assumptions of traditional plan-driven or agile methodologies. This research paper's methodology follows the guidelines of design science experts [21, 25, 40] by basing both the motivation for, and the nature of, the methodology design on interdependence theory [49] and services science [14], providing principles of implementation, offering testable propositions for future research, and demonstrating its effectiveness. Although SoMSD is not without its limitations and assumptions, this research can inspire new ideas and designs in the application of service-oriented concepts to the software development methodology domain. Future research should examine SoMSD adoption considerations and further develop the principles for implementation.

NOTES

1. This approach does not refer to a methodology for creating service-oriented software. Rather, it is the methodology itself that is service oriented and not necessarily the software being developed. In the case of SoMSD, the actual system being developed may take any architectural form.

2. It could be argued that the various SoSE methodologies are also "service oriented" in that the subteams are grouped based on the technical services they are tasked to develop. However, SoMSD implements such teams for any type of architecture development and also formalizes the human service definitions to a much greater extent for future reuse as discussed later.

3. The modularization of a project process is not only based on dividing processes with sequential versus reciprocal interdependencies. As implied in Figure 1, by having multiple sections of the same interdependency type in a row, further modularization will also take place along other traditional, natural break points. However, when implementing SoMSD, the interdependency is another major factor to consider when modularizing the process.

4. It is important to note that each process may have multiple legitimate service patterns. The "best" pattern in this scenario may be determined by factors such as the availability of resources, need for low cost versus quick delivery, and so forth.

5. In practice, actual service patterns will likely be much more granular and define services at a more detailed level. However, the example demonstrates how each service may have mul-

multiple potential subservice patterns just as each software development project may have multiple potential service patterns.

6. Notice that this relationship is similar to that of loosely coupled Web services in SOA-based systems. Web services do not need to understand the inner workings of other Web services in order to exchange data. SoMSD is similar except that instead of passing data between functions, workflow is being passed between human service providers.

7. In the business process modeling context, “AS-IS” refers to the creation of business process models of the *existing* systems and “TO-BE” refers to the creation of models of how the new system *should* be.

8. For clarity, the remainder of the paper will refer to the typical AS-IS and TO-BE process models created for each software project as “business process models” and the models created specifically about the project process (SoMSD service patterns) will be referred to as “project process models.”

9. Scope changes could not be used here because they were not formally allowed at this point in the project. However, informally, requirements still fluctuated greatly during implementation.

10. Several techniques for establishing formative measurement validity have been outlined in information systems research [36]. The purpose of these techniques is to ensure that the measurement items used best represent the real content and understanding of the factor being measured. Establishing content validity is of primary importance with formative constructs [51]. Therefore, we validated our formative measures by discussing the meaning behind each variable with EPC’s PMs and unit director, as well as those of two other large, comparable organizations in the area, and then jointly agreed on the perceptual measurement items that would best capture the differences in the variables discussed above among the projects.

REFERENCES

1. Baker, A.L., and Zweben, S.H. Use of software science in evaluating modularity concepts. *IEEE Transactions on Software Engineering*, 5, 2 (1979), 110–120.
2. Baldwin, C.Y., and Clark, K.B. *Design Rules: The Power of Modularity*. Cambridge: MIT Press, 2000.
3. Barki, H.; Rivard, S.; and Talbot, J. An integrative contingency model of software project risk management. *Journal of Management Information Systems*, 17, 4 (Spring 2001), 37–69.
4. Barlow, J.B.; Giboney, J.S.; Keith, M.J.; Wilson, D.W.; Schuetzler, R.M.; Vance, A.; and Lowry, P.B. Overview and guidance on agile development in large organizations. *Communications of the Association for Information Systems*, 29, 2 (2011), 25–44.
5. Benbasat, I.; Goldstein, D.K.; and Mead, M. The case research strategy in studies of information systems. *MIS Quarterly*, 11, 3 (1987), 369–386.
6. Bieberstein, N.; Bose, S.; Walker, L.; and Lynch, A. Impact of service-oriented architecture on enterprise systems, organizational structures, and individuals. *IBM Systems Journal*, 44, 4 (2005), 691–708.
7. Boehm, B., and Bhuta, J. Balancing opportunities and risks in component-based software development. *IEEE Software*, 25, 6 (2008), 56–63.
8. Boehm, B., and Turner, R. Using risk to balance agile and plan-driven methods. *Computer*, 36, 6 (2003), 57–66.
9. Burns, T., and Stalker, G.M. *The Management of Innovation*. London: Tavistock, 1961.
10. Chesbrough, H., and Spohrer, J. A research manifesto for services science. *Communications of the ACM*, 49, 7 (2006), 35–40.
11. Cockburn, A., and Highsmith, J. Agile software development: The people factor. *Computer*, 34, 11 (2001), 131–133.
12. Dennis, A.R.; Fuller, R.M.; and Valacich, J.S. Media, tasks, and communication processes: A theory of media synchronicity. *MIS Quarterly*, 32, 3 (2008), 575–600.
13. Erl, T. *Service-Oriented Architecture: Concepts, Technology, and Design*. Upper Saddle River, NJ: Prentice Hall, 2005.
14. Foster, I. Service-oriented science. *Science*, 308, 5723 (May 6, 2005), 814–817.

15. Galbraith, J.R. *Designing Complex Organizations*. Reading, MA: Addison-Wesley, 1973.
16. Gemino, A.; Reich, B.H.; and Sauer, C. A temporal model of information technology project performance. *Journal of Management Information Systems*, 24, 3 (Winter 2007–8), 9–44.
17. Gholami, M.F.; Sharifi, M.; and Jamshidi, P. Enhancing the OPEN process framework with service-oriented method fragments. *Software and Systems Modeling* (November 2011), 1–30.
18. Goatham, R. Why do projects fail? CalleadConsulting, July 22, 2011 (available at http://callead.com/WTPF/?page_id=2213/).
19. Golpayegani, S.A.H., and Emamizadeh, B. Designing work breakdown structures using modular neural networks. *Decision Support Systems*, 44, 1 (2007), 202–222.
20. Gregor, S. The nature of theory in information systems. *MIS Quarterly*, 30, 3 (2006), 611–642.
21. Gregor, S., and Jones, D. The anatomy of a design theory. *Journal of the Association for Information Systems*, 8, 5 (2007), 312–335.
22. Gruman, G. Breaking SOA bottlenecks. *InfoWorld* (May 14, 2007) (available at www.infoworld.com/t/architecture/breaking-soa-bottlenecks-068/).
23. Hansen, B. Experience with modular concurrent programming. *IEEE Transactions on Software Engineering*, 3, 2 (1977), 156–159.
24. Hardgrave, B.C.; Davis, F.D.; and Riemenschneider, C.K. Investigating determinants of software developers' intentions to follow methodologies. *Journal of Management Information Systems*, 20, 1 (Summer 2003), 123–151.
25. Hevner, A.R.; March, S.T.; Park, J.; and Ram, S. Design science in information systems research. *MIS Quarterly*, 28, 1 (2004), 75–105.
26. Highsmith, J., and Cockburn, A. Agile software development: The business of innovation. *Computer*, 34, 9 (2001), 120–122.
27. Hillegersberg, J.; Moonen, H.; and Dalmolen, S. Coordination as a service to enable agile business networks. In J. Kotlarsky, I. Oshri, and L.P. Willcocks (eds.), *The Dynamics of Global Sourcing: Perspectives and Practices*. Berlin: Springer, 2012, pp. 164–174.
28. Jarvis, C.B.; MacKenzie, S.B.; and Podsakoff, P.M. A critical review of construct indicators and measurement model misspecification in marketing and consumer research. *Journal of Consumer Research*, 30, 2 (2003), 199–218.
29. Kearns, G., and Sabherwal, R. Strategic alignment between business and information technology: A knowledge-based view of behaviors, outcome, and consequences. *Journal of Management Information Systems*, 23, 3 (Winter 2006–7), 129–162.
30. Kraut, R.E., and Streeter, L.A. Coordination in software development. *Communications of the ACM*, 38, 3 (1995), 69–81.
31. Kumaran, S.; Bishop, P.; Chao, T.; Dhoolia, P.; Jain, P.; Jaluka, R.; Ludwig, H.; Moyer, A.; and Nigam, A. Using a model-driven transformational approach and service-oriented architecture for service delivery management. *IBM Systems Journal*, 46, 3 (2007), 513–529.
32. Langlois, R.N. Modularity in technology and organization. *Journal of Economic Behavior & Organization*, 49, 1 (2002), 19–37.
33. MacKenzie, C.M.; Laskey, K.; McCage, F.; Brown, P.R.; and Metz, R. Oasis reference model for service oriented architecture 1.0. OASIS Standard, October 12, 2006 (available at <http://docs.oasis-open.org/soa-rm/v1.0/soa-rm.pdf>).
34. Malone, T.W. Modeling coordination in organizations and markets. *Management Science*, 33, 10 (1987), 1317–1332.
35. Malone, T.W., and Crowston, K. The interdisciplinary study of coordination. *ACM Computing Surveys*, 26, 1 (1994), 87–119.
36. Marakas, G.M.; Johnson, R.D.; and Clay, P.F. The evolving nature of the computer self-efficacy construct: An empirical investigation of measurement construction, validity, reliability and stability over time. *Journal of the Association for Information Systems*, 8, 1 (2007), 15–46.
37. Nerur, S.; Mahapatra, R.; and Mangalaraj, G. Challenges of emigrating to agile methodologies. *Communications of the ACM*, 48, 5 (2005), 72–78.
38. Nidumolu, S. The effect of coordination and uncertainty on software project performance—Residual performance risk as an intervening variable. *Information Systems Research*, 6, 3 (1995), 191–219.

39. Nowicki, E., and Smutnicki, C. A fast taboo search algorithm for the job shop problem. *Management Science*, 42, 6 (1996), 797–813.
40. Peffers, K.; Tuunanen, T.; Rothenberger, M.A.; and Chatterjee, S. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24, 3 (Winter 2007–8), 45–77.
41. Rodriguez-Martinez, L.; Mora, M.; Alvarez, F.; Garza, L.; Duran, H.; and Munoz, J. Review of relevant system development life cycles in service-oriented software engineering. *Journal of Applied Research and Technology*, 10, 2 (2012), 94–113.
42. Schilling, M.A. Toward a general modular systems theory and its application to interfirm product modularity. *Academy of Management Review*, 25, 2 (2000), 312–334.
43. Schmidt, R.; Lyytinen, K.; Keil, M.; and Cule, P. Identifying software project risks: An international Delphi study. *Journal of Management Information Systems*, 17, 4 (Spring 2001), 5–36.
44. Schwalbe, K. *Information Technology Project Management*. Boston: Course Technology, 2010.
45. Sheth, A.; Verma, K.; and Gomadam, K. Semantics to energize the full services spectrum. *Communications of the ACM*, 49, 7 (2006), 55–61.
46. Shlomo, G. Impact of various work-breakdown structures on project conceptualization. *International Journal of Project Management*, 12, 3 (1994), 165–171.
47. Sparrowe, R.T.; Liden, R.C.; Wayne, S.J.; and Kraimer, M.L. Social networks and the performance of individuals and groups. *Academy of Management Journal*, 44, 2 (2001), 316–325.
48. Sutherland, J.; Viktorov, A.; Blount, J.; and Puntikov, N. Distributed scrum: Agile project management with outsourced development teams. In R.H. Sprague (ed.), *Proceedings of the 40th Annual Hawaii International Conference on System Sciences*. Los Alamitos, CA: IEEE Computer Society Press, 2007.
49. Thompson, J.D. *Organizations in Action: Social Science Bases of Administrative Theory*. New York: McGraw-Hill, 1967.
50. Tiwana, A., and Mclean, E.R. Expertise integration and creativity in information systems development. *Journal of Management Information Systems*, 22, 1 (Summer 2003), 13–43.
51. Tsai, W.T. Service-oriented system engineering: A new paradigm. In *IEEE International Workshop on Service-Oriented System Engineering, 2005*. Los Alamitos, CA: IEEE Computer Society Press, 2005, pp. 3–6.
52. Tu, Q.; Vonderembse, M.A.; Ragu-Nathan, T.S.; and Ragu-Nathan, B. Measuring modularity-based manufacturing practices and their impact on mass customization capability: A customer-driven perspective. *Decision Sciences*, 35, 2 (2004), 147–168.
53. Turk, D.; France, R.; and Rumpe, B. Assumptions underlying agile software-development processes. *Journal of Database Management*, 16, 4 (2005), 62–87.
54. Tushman, M.L., and Nadler, D.A. Information processing as an integrating concept in organizational design. *Academy of Management Review*, 3, 3 (1978), 613–624.
55. Uschold, M., and Gruninger, M. Ontologies: Principles, methods and applications. *Knowledge Engineering Review*, 11, 2 (1996), 93–136.
56. Vargo, S.L., and Lusch, R.F. Evolving to a new dominant logic for marketing. *Journal of Marketing*, 68, 1 (2004), 1–17.
57. Vidgen, R., and Wang, X.F. Coevolving systems and the organization of agile software development. *Information Systems Research*, 20, 3 (2009), 355–376.
58. Vinekar, V.; Slinkman, C.W.; and Nerur, S. Can agile and traditional systems development approaches coexist? An ambidextrous view. *Information Systems Management*, 23, 3 (2006), 31–42.
59. Wagner, J.A. Organizations. In A.E. Kazdin (ed.), *Encyclopedia of Psychology*. Washington, DC: American Psychological Association, 2000, pp. 14–20.
60. Wasserman, A.I. Toward a discipline of software engineering. *IEEE Software*, 13, 6 (1996), 23–31.
61. Xia, W.D., and Lee, G.H. Complexity of information systems development projects: Conceptualization and measurement development. *Journal of Management Information Systems*, 22, 1 (Summer 2005), 45–83.

